Práctica 5: Sincronización y uso compartido de recursos

Rodrigo García Carmona
rodrigo.garcia@upm.es



Esta práctica es individual y sirve tanto para desarrollar como para poner a prueba los conocimientos adquiridos. Realizar esta práctica es recomendable para aumentar las probabilidades de éxito en los exámenes teóricos.

1. Preparación de la práctica

El objetivo de esta práctica es aprender a usar los mecanismos que permiten sincronizar diferentes hebras y que estas comparan los mismos recursos sin que se produzcan problemas. Se estudiarán los siguientes conceptos:

- Uso de monitores para la sincronización de hebras.
- Uso de colas para evitar la inanición de hebras.

Trabajo previo al laboratorio

El tiempo de las clases de laboratorio es muy valioso, ya que en ellas tendrá a su disposición a los profesores para resolver cualquier duda que le pueda surgir. Por tanto, para facilitar al máximo el aprovechamiento de este tiempo, las prácticas se han dividido en varias secciones. La primera de ellas ("preparación de la práctica") puede ¡y debe! realizarse **antes de la sesión de laboratorio** propiamente dicha. Leer esta sección antes de entrar a clase hará que su tiempo en el laboratorio sea más productivo y provocará que las dudas más importantes le surjan **durante la clase**, y **no después**, cuando se encuentre estudiando solo en su casa y la única opción que le quede sea escribir un email al profesor.

De hecho, es especialmente importante evitar que se den situaciones como esta la noche antes de un examen.

1.1. Sistemas complejos

En la práctica 4 aprendió qué son los sistemas complejos y en qué se diferencian de los sistemas simples. Entendió que están formados por un conjunto de agentes que interactúan entre sí y que, como consecuencia de estas interacciones, aparecen comportamientos emergentes, cuya predicción es imposible salvo mediante simulaciones. También implementó un ejemplo paradigmático de sistema complejo: el servicio de Urgencias de un hospital.

O eso creía usted, porque en realidad le he contado una mentira: en la práctica 4 no llegó a implementar un sistema complejo.

...

Por favor, no se enfade. No se sienta engañado ni piense que todo su esfuerzo ha sido en vano, ya que hacer la práctica 4 es un paso previo imprescindible para implementar un verdadero sistema complejo: el de esta práctica, la 5.

1.1.1. Interacciones entre agentes

Para que un sistema pueda considerarse complejo, no solo es necesario que cuente con varios agentes que operen de forma independiente, sino también que estos **interactúen entre sí**.

Esto en la práctica 4 no ocurría. Si se fija, es irrelevante cuántos pacientes estén pasando por el servicio de Urgencias, ya que ninguno de ellos tendrá que esperar a que otro haya sido atendido. Además, no existe ningún control sobre cuántos pacientes se encuentran a la vez en un servicio determinado. Se ha tratado al hospital como si tuviera capacidad infinita, cuando, claramente, esto no puede ser así.

Si aún duda de esto (la negación es siempre el paso previo a la aceptación), piense en si el recorrido que hace cada paciente o el tiempo que tarda en ser atendido varía en función de cuándo entra en el hospital o de cuántos pacientes han llegado antes que él. La respuesta a esta pregunta es que no varía; podrían ejecutarse todos los pacientes a la vez o uno tras otro y no habría diferencia alguna en el resultado desde el punto de vista de cada paciente.

Vamos a solucionar esto.

1.2. Servicio de Urgencias 2.0

Para que el simulador del servicio de Urgencias sea un verdadero sistema complejo y represente de una forma más fidedigna lo que ocurre en un hospital, vamos a modificar la clase `Area` (que modela una sección del servicio de Urgencias), de tal forma que cada sección solo pueda atender a un número determinado de pacientes al mismo tiempo. De esta forma, si un `Area` concreta está llena, el resto de pacientes que quieran acceder a ella tendrán que esperar hasta que se haya terminado de atender a alguno de los pacientes actuales.

Esta modificación puede verse en el diagrama de clases de la Figura 1, que también incluye, por comodidad, el resto de clases que modelan el servicio de Urgencias, y que no han variado (en lo que a atributos y métodos respecta) desde la práctica 4:

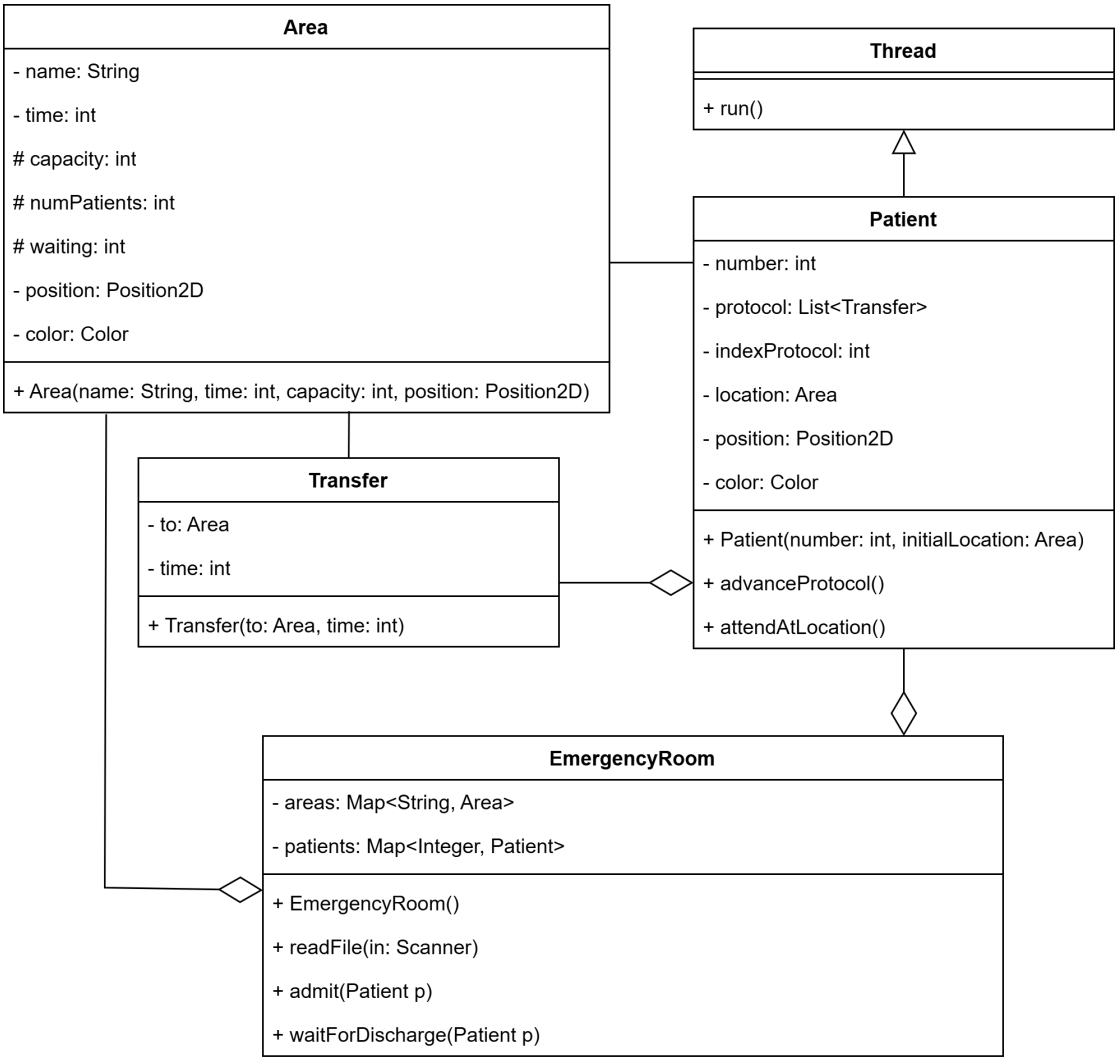


Figura 1: Diagrama de clases del simulador del servicio de Urgencias 2.0.

Fíjese en que ahora `Area` cuenta con tres atributos nuevos:

- `capacity`: Un `int` que representa el número máximo de pacientes que pueden atenderse en esa sección al mismo tiempo. Este valor se especifica en el momento de creación del `Area` y, como podrá observar, el constructor de la clase ha cambiado para incluir también este atributo.
- `numPatients`: Un `int` que indica cuántos pacientes están siendo atendidos en este momento. Este número siempre debe ser igual o inferior a `capacity`. Cuando se crea un `Area` nueva este número es 0, y nunca puede ser inferior a este valor (no se puede estar atendiendo a un número negativo de pacientes).
- `waiting`: Un `int` que almacena el número de pacientes que están esperando a ser atendidos en esta sección. Este número puede ser tan grande como sea necesario; es decir, que no hay límite al número de pacientes que pueden estar esperando para entrar. Cuando se crea un `Area` nueva este número es 0, y nunca puede ser inferior a este valor (no puede hacer un número negativo de pacientes esperando).

Observe el código de la clase `Area` (está en el paquete `es.upm.dit.aled.lab5`) que se le proporciona y fíjese bien en las partes que tendrá que implementar (las marcadas como `TODO`). Tenga en cuenta que, a diferencia de en prácticas anteriores, en esta tendrá que implementar los métodos por completo, incluyendo su cabecera (en lugar de únicamente el cuerpo).

Responda a las siguientes preguntas:

- ¿Puede el atributo `capacity` de `Area` cambiar durante la ejecución del programa? Si es así, ¿en qué circunstancias?, ¿cuánto se incrementará o reducirá cada vez?
- ¿Puede el atributo `numPatients` de `Area` cambiar durante la ejecución del programa? Si es así, ¿en qué circunstancias?, ¿cuánto se incrementará o reducirá cada vez?
- ¿Puede el atributo `waiting` de `Area` cambiar durante la ejecución del programa? Si es así, ¿en qué circunstancias?, ¿cuánto se incrementará o reducirá cada vez?

1.2.1. Monitores

En la programación concurrente, un monitor es una clase que encapsula y protege todos los accesos (tanto lecturas como escrituras) a un recurso compartido. En lo que a esta práctica respecta, vamos a tener un monitor: la clase `Area`, que posee varios atributos que son susceptibles de ser modificados por varios agentes (los pacientes) al mismo tiempo. Por tanto, la clase `Area` deberá proteger estos atributos, asegurándose de que hay exclusión mutua en todos los fragmentos de código en los que se accede a ellos. Recuerde que estos fragmentos en los que se accede a recursos compartidos se denominan "regiones críticas".

Para implementar un monitor en Java, lo que se debe hacer es declarar los métodos que contengan regiones críticas como sincronizados.

Responda a las siguientes preguntas:

- ¿Qué atributos de `Area` son compartidos?
- ¿Qué métodos de `Area` contienen regiones críticas?
- ¿Cómo se indica en Java que un método es sincronizado?

1.2.2. Nuevo formato de archivos

Ahora abra el archivo `er_data.txt`, que se encuentra en la carpeta `er`. Este archivo contiene la información necesaria para simular un servicio de Urgencias concreto. Aunque es muy parecido al de la práctica 4, contiene una diferencia importante. Fíjese en la información relativa a las secciones:

```
AREA  80  80 1000 2 Admision
AREA 160 320 1500 3 Triaje
AREA 160 200 3000 10 SalaEspera
AREA 320  80 3000 2 Box_Trauma
...
```

Estas líneas ahora contienen un cuarto número entero, que indica cuántos pacientes puede estar atendiendo al mismo tiempo cada sección (es decir, su `capacity`). En este ejemplo, la sección "Triaje" podrá aceptar hasta a 3 pacientes al mismo tiempo.

Si observa el código del método `readFile(Scanner in)` de la clase `EmergencyRoom`, que es el que procesa estos archivos, verá que ha sido modificado respecto al de la práctica 4 para tener esto en cuenta:

```
...
int capacity = in.nextInt();
...
Area area = new Area(name, time, capacity, new Position2D(x, y));
...
```

Por lo demás, `EmergencyRoom` no ha experimentado ningún otro cambio. Lo mismo puede decirse de `Transfer` y `Simulation`.

1.2.3. Nueva interfaz gráfica

Dentro del paquete `es.upm.dit.aled.lab5.gui` se encuentran las clases necesarias para ejecutar una interfaz gráfica que permite visualizar el movimiento de los pacientes por el servicio de Urgencias. Respecto a la práctica 4, solo se ha modificado la clase `EmergencyRoomGUI`, que ahora muestra en la caja correspondiente a cada sección cuántos pacientes está atendiendo en este momento, cuántos pacientes puede atender en total, y cuántos pacientes están esperando.

La Figura 2 muestra una captura de esta interfaz gráfica en la que se puede ver que "Box Trauma" está atendiendo a 1 paciente cuando su capacidad total es de 2 pacientes, mientras que "Box Criticos" está llena (solo admite a 1 paciente) y tiene a 5 pacientes esperando.

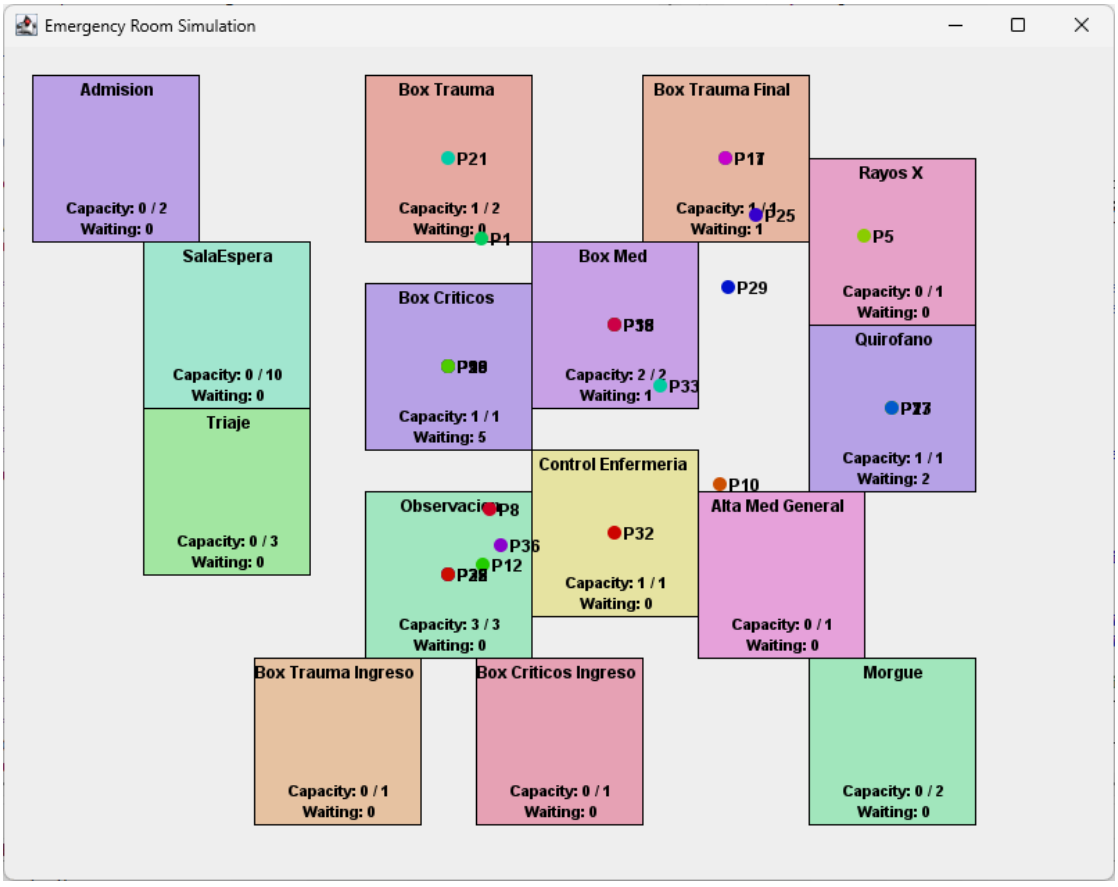


Figura 2: Simulación del servicio de Urgencias.

Pero no se preocupe, porque nada de esto le afecta. `EmergencyRoomGUI` sigue teniendo los mismos métodos que antes y estos se utilizan de la misma forma.

Patrón singleton

Sí, `EmergencyRoomGUI` sigue siendo un *singleton*; no se olvide de usar `getInstance()`.

1.3. Objetivo y descarga de la práctica

El objetivo de esta práctica es mejorar el programa de simulación de un servicio de Urgencias, de manera que tenga en cuenta la capacidad de cada una de las secciones de dicho servicio. Para ello tendrá que implementar un monitor que controle el acceso a los recursos compartidos (las secciones) por parte de los agentes (los pacientes).

La práctica está disponible en GitHub, en la siguiente dirección de Internet (URI):
https://github.com/rgarciacarmona/ALED-lab5
Para trabajar en ella, siga estos pasos:

1. Haga un *fork* de este repositorio a su cuenta de GitHub.
2. Antes de empezar a hacer *commits*, compruebe en *Settings* → *General* de su *fork* que la visibilidad es **pública**. Solo se evaluarán los *commits* hechos sobre un *fork* público.
3. Clone su *fork* e impórtelo en Eclipse. Para ello, seleccione *File* → *Import...* y, de entre las opciones disponibles, *Git* → *Projects from Git* → *Clone URI*. A continuación, introduzca la URI de su *fork* (`https://github.com/`*usuario*`/ALED-lab5`, donde *usuario* es su nombre de usuario de GitHub), no la del repositorio original. Después, haga click en *Next* tantas veces como sea necesario y, por último, en *Finish*.
4. Trabaje, haga *commit* de sus cambios y haga *push* a su *fork*.

Una vez descargado el código de la práctica, verá que este está compuesto de dos paquetes:

- `es.upm.dit.aled.lab5`: Que corresponde al código que simula el servicio de Urgencias.
- `es.upm.dit.aled.lab5.gui`: Que contiene las clases que permiten visualizar el movimiento de los pacientes y la ocupación de las secciones en tiempo real.

Enhorabuena, ya ha terminado con el trabajo previo a la sesión de laboratorio. Las secciones de la 2 en adelante puede realizarlas en clase.

2. Control de acceso a secciones

Antes de seguir, asegúrese de que le queda meridianamente claro que:

- Los objetos de la clase `Area` contienen recursos compartidos, y por tanto el código de esta clase debe encargarse de proteger su acceso. Este es el **monitor**.
- Los objetos de la clase `Patient` van a intentar utilizar los recursos compartidos, usando para ello los métodos de `Area`. Estos son los **agentes**.

Con esto en mente, primero implementará los métodos que faltan del monitor y, posteriormente, modificará el comportamiento de los agentes para que usen dichos métodos de la forma correcta.

2.1. Implementación del monitor

Si observa el código fuente de `Area` verá que contiene los comentarios de varios métodos que no solo no están implementados, sino tampoco definidos (no está escrita su cabecera). Su primera tarea será, en base a los comentarios antes mencionados, definir (de nuevo, escribir las cabeceras) de los métodos siguientes:

- `getCapacity`: Devuelve el valor actual de `capacity`.
- `getNumPatients`: Devuelve el valor actual de `numPatients`.

- `getWaiting`: Devuelve el valor actual de `waiting`.
- `enter`: Un `Patient` que esté intentando entrar en esta `Area` para ser atendido llamará a este método.
- `exit`: Un `Patient` que ya haya sido atendido llamará a este método antes de avanzar al siguiente `Transfer` de su protocolo.

Los comentarios le indican qué parámetros y qué tipos de retorno deben tener los métodos, si es que no los ha deducido usted mismo.

Importante: Asegúrese de que los métodos que vayan a contener regiones críticas estén sincronizados.

Sincronización también para lectura

No se olvide de que el código que accede a un recurso compartido solo para leerlo (sin modificarlo) también es una sección crítica. Esto se debe a que, aunque la hebra que lee el recurso no lo modifique ella misma (no puede haber colisión de escrituras), eso no quiere decir que dicho recurso no esté siendo modificado al mismo tiempo por otro agente distinto (sí puede haber colisión de escritura-lectura).

A continuación, termine de implementar el constructor de `Area`, pues solo contiene el código en el que se le asigna el `color` por defecto.

Ahora implemente los métodos que definió antes. `getCapacity`, `getNumPatients` y `getWaiting` no tienen más misterio, pues son accesores normales y corrientes, pero los métodos `enter` y `exit` requieren de algo más de reflexión.

2.1.1. Método `enter`

Cuando un `Patient` llama a este método, eso quiere decir que está intentando acceder a un recurso compartido. Esto solo será posible si ahora mismo el número de pacientes que están siendo atendidos en esa sección es inferior a la capacidad de la misma. Por tanto, el método `enter` deberá contener un bucle `while` que compruebe esta condición y, de no cumplirse, haga a la hebra del paciente esperar mediante una llamada a `wait()`.

Además, `enter` también deberá encargarse de modificar los atributos `numPatients` y `waiting`. Intente pensar usted mismo cuánto deberían incrementarse y/o decrementarse y en qué momento. Si no consigue apañarse, un poco más abajo tiene la solución.

...

¿Seguro que no se le ocurre? Piense un poco más...

Venga, se lo cuento, pero solo para que compruebe que la solución que se le había ocurrido a usted es la correcta. Estos dos atributos deberán modificarse de la siguiente forma:

- Si un paciente se ve forzado a esperar porque la sección ya está llena, `waiting` deberá incrementarse en 1.
- Si un paciente logra acceder a la sección, `numPatients` deberá incrementarse en 1.
- Si un paciente que estaba esperando logra acceder a la sección, `numWaiting` deberá reducirse en 1.

El bucle while

Es **muy importante** que la comprobación de la condición de acceso a la sección se haga dentro de un bucle `while`, y no solo una única vez. La razón es que, incluso aunque una hebra se haya despertado tras un `wait()`, eso no asegura que la condición se cumpla ahora; otra hebra podría haber modificado el recurso compartido en el tiempo que pasa desde que se despierta a la hebra hasta que esta se ejecuta. O puede incluso que la hebra haya sido despertada por una razón distinta a que la condición que la hizo esperar se cumpla.

2.1.2. Método `exit`

Este método deberá indicar que se ha liberado una plaza en el `Area` (modificando `numPatients`) y despertar a todas las hebras de los `Patients` que estaban esperando para acceder. Esto último puede hacerse mediante una llamada al método `notifyAll()`.

¡Pero si no necesito al paciente!

Es posible que al implementar los métodos `enter` y `exit` se haya dado cuenta de que, a pesar de que ambos tienen como parámetro un objeto `Patient`, luego este no necesitan usarlo para nada. La razón de esto es que tener la referencia al paciente que está intentando entrar o salir de la sección en estos métodos es muy útil, ya que puede usarse para imprimir por pantalla mensajes de información que indiquen en qué estado se encuentra cada `Patient` (esperando para entrar, siendo atendido, saliendo...).

Además, en la sección 3 verá que sí que necesita al paciente para el método `enter`.

2.2. Modificación de `Patient`

Ahora que ha terminado de implementar el monitor, es necesario que modifique el comportamiento de `Patient` (su método `run()`) para que haga uso de los métodos `enter` y `exit` que acaba de escribir. Los pasos que debe seguir cada paciente ahora son:

1. Intentar acceder a la ubicación actual (`enter`).
2. Ser atendido en la ubicación actual.
3. Salir de la ubicación actual (`exit`).
4. Avanzar al siguiente paso en su protocolo.
5. Volver al paso 1.

Para los detalles sobre los puntos 2, 4 y 5, consulte la práctica 4. Idealmente, no debería necesitar programarlos, pues ya escribió en dicha práctica el código que hacía esto.

2.3. Ejecución de la simulación

Pues ya estaría. Solo le queda ejecutar el método `main()` de la clase `Simulation`. No se olvide de crear una configuración de lanzamiento que pase como primer argumento la ubicación del archivo de texto que contiene los datos del servicio de Urgencias: `er_data.txt`, que se encuenta en la carpeta `er`. Si todo va bien, debería ver como los pacientes esperan pacientemente (jeje) a ser atendidos, como en la Figura 2.

3. Control de acceso con lista ordenada

Su servicio de Urgencias tiene un problema: no se atiende a los pacientes en orden de llegada. Cuando un `Patient` sale de un `Area`, despierta a todos los que estaban esperando (mediante `notifyAll()`). En ese momento se produce una suerte de "estampida virtual", en la que todos los pacientes intentan entrar corriendo en la sección, algo que solo conseguirá el que sea más rápido (o tenga más suerte). Esto podría desembocar en situaciones que en programación concurrente se llaman de "inanición", en las que un `Patient` nunca logra entrar en un `Area` porque continúan llegando pacientes nuevos que se le "cuelan"

3.1. Clase `AreaQueue`

Para evitar esto va a implementar un versión mejorada de `Area`, llamada `AreaQueue`, que utiliza una cola para respetar el orden de llegada de los pacientes. La Figura 3 le muestra lo que necesita saber de esta clase para poder implementarla. Como puede observar, esta nueva clase extiende a `Area`, por lo que hereda todos los métodos y atributos de esta. Eso incluye los métodos que usted mismo programó en la sección 2. Solo añade un constructor y reimplementa (sobreescribe usando `@Override`) el método `enter` de su clase madre.

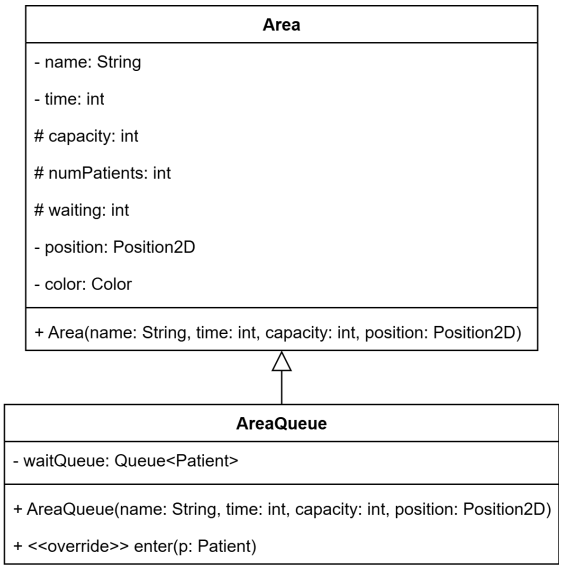


Figura 3: Diagrama de clases de `AreaQueue`.

En esta sección implementará por completo esta clase. Sí, desde cero; esta vez no se le proporciona un esqueleto que tendrá que rellenar. A estas alturas del curso ya es usted un programador hecho y derecho, por lo que no tendrá ningún problema. Además, esta tarea no es tan laboriosa como en un principio podría parecer, ya que al ser `AreaQueue` hija de la clase `Area`, la mayor parte del código de la clase ya está implementado (se hereda de `Area`). De hecho, fíjese otra vez en la Figura 3; verá que en en la clase `AreaQueue` solo aparecen indicados dos métodos y un atributo. Estas tres cosas es lo que tiene que implementar.

Protected

Quizá haya observado que en la clase `Area` hay tres atributos que no son `private`, sino `protected`: `capacity`, `numPatients` y `waiting`. Si revisa sus apuntes de Fundamentos de Programación verá que `protected` es un tipo de visibilidad que se usa precisamente en situaciones como esta, en las que queremos que las clases hijas (o en el mismo paquete) puedan acceder a los atributos, pero que estos sigan siendo privados para clases que se encuentren en otros paquetes.

Fíjese en que el método `enter`, aunque ya estaba presente en la clase `Area`, ahora vuelve a aparecer en `AreaQueue`, pero precedido del texto `«override»`. Este texto indica que la clase tiene que reimplementar un método que ya poseía la clase madre, pues la funcionalidad ha cambiado. Y es que ahora ya no es suficiente con comprobar que se cumple la condición de ocupación (que `numPatients` sea menor que `capacity`), sino que también es necesario que el paciente que quiere entrar sea **el primero de la cola de espera**. Esta cola de espera está representada por el atributo `waitQueue`.

A la hora de implementar `AreaQueue`, tenga en cuenta los siguientes consejos:

- `Queue` no es una clase, sino un interfaz. Por tanto, no puede crear directamente objetos de `Queue`, sino que tendrá que usar su implementación: `LinkedList`.
- Tendrá que utilizar los siguientes métodos del interfaz `Queue`:
  - `add(Patient p)`: Añade a `p` al **final** de la cola.

- `remove()`: Devuelve el `Patient` que se encuentra al **principio** de la cola y lo elimina, de tal forma que el siguiente elemento de la cola pasa a ser el primero.
- `peek()`: Devuelve el `Patient` que se encuentra al **principio** de la cola. La diferencia con `remove()` es que `peek()` no elimina (saca) al elemento de la cola, simplemente permite ver cuál es.

- Para indicar que el método `enter` reimplementa el de la clase madre, tendrá que usar la anotación Java `@Override`:

```
@Override
public synchronized void enter(Patient patient) {
...
}
```

3.2. Ejecución de la simulación

Ya solo queda ejecutar la simulación. Pero, para esto, antes debe modificar una línea de la clase `EmergencyRoom` para que utilice la clase `AreaQueue` que acaba de crear, en lugar de `Area`. Sustituya el código que se encuentra en la línea 97 de `EmergencyRoom` por el siguiente:

```
Area area = new AreaQueue(name, time, capacity, new Position2D(x, y));
```

Por último, ejecute de nuevo la simulación de la sección 2.3. Si todo ha ido bien, verá como los pacientes son atendidos en orden de llegada a cada zona del servicio de Urgencias.

Enhorabuena. Ha terminado la práctica.

4. Extras

Si siente curiosidad, pruebe a llevar a cabo las siguientes modificaciones del proyecto. ¡Ojo!, las dos últimas implican un trabajo de desarrollo considerable.

- Modifique el código para que tanto el tiempo de atención de un `Area` como el tiempo de traslado de un `Transfer` sean tiempos medios. El tiempo real se calculará de forma aleatoria, estando comprendido entre `time-1000` milisegundos y `time+1000` milisegundos.
- Añada al proyecto un nuevo agente: el `Doctor`. `Doctor` estará asignado a un conjunto de `Areas` e irá atendiendo a los pacientes que acudan a dichas secciones. Si realiza esta modificación, las `Area` ya no tendrán `capacity` ni `numPatients`, pues cuántos pacientes estén siendo atendidos dependerá de qué `Doctors` están en dicha `Area`. De hecho, ya no son las `Areas` las que "atienden", sino los `Doctors`.
- Modifique el proyecto para que, cuando un paciente tiene que esperar, vaya al `Area` cuyo nombre sea "SalaEspera". Si un `Patient` tiene que entrar en "SalaEspera" y esta está llena, en lugar de eso abandonará el hospital (o, si lo prefiere, morirá, que es más dramático). Asuma que el tiempo necesario para trasladarse a "SalaEspera" cuando este traslado no es parte del protocolo (un `Transfer`) es siempre de 1000 milisegundos.